

Boolean Matrixmultiplikation in $O(n^2)$

Anwendung des Signatur-Verfahrens aus dem Subgraph Algorithmus

Stephan Epp

16. Januar 2026

Inhaltsverzeichnis

1	Einführung	2
2	Algorithmus	3
2.1	Implementierung	4
2.2	Beispiel	6
3	Vergleich herkömmlicher Methoden	7
4	Erweiterung auf k-beschränkte Werte	7
4.1	Algorithmus in $O(k \cdot n^2)$	8
5	Zusammenfassung	8
5.1	Anwendungen	9
5.2	Implementierung	9

1 Einführung

Diese Arbeit beschreibt eine Anwendung der Signatur-Technik aus dem Subgraph Algorithmus auf die Boolean Matrixmultiplikation. Während die allgemeine Matrixmultiplikation eine Komplexität von mindestens $\Omega(n^{2.37})$ hat, wird gezeigt, dass Boolean Matrixmultiplikation mit der Signatur-Methode in $O(n^2)$ Zeit berechnet werden kann. Dies wird durch geschickte Nutzung von Bitoperationen und polynomialer Hash-Kodierung erreicht.

Die klassische Matrixmultiplikation zweier $n \times n$ Matrizen A und B berechnet eine Ergebnismatrix C mit:

$$C_{ij} = \sum_{k=1}^n A_{ik} \cdot B_{kj} \quad (1)$$

Die naive Implementierung hat eine Laufzeit von $O(n^3)$. Fortgeschrittene Algorithmen wie STRASSEN (1969) oder COPPERSMITH-WINOGRAD (1990) erreichen $O(n^{2.807})$ bzw. $O(n^{2.376})$.

Für **Boolean Matrizen**, bei denen alle Einträge $\{0, 1\}$ sind und die Operationen durch logische Operationen ersetzt werden, kann die Signatur-Technik aus dem Subgraph Algorithmus genutzt werden, um eine Laufzeit von $O(n^2)$ zu erreichen.

Definition 1 (Boolean Matrixmultiplikation). Für zwei Boolean Matrizen $A, B \in \{0, 1\}^{n \times n}$ ist die Boolean Matrixmultiplikation definiert als:

$$C_{ij} = \bigvee_{k=1}^n (A_{ik} \wedge B_{kj})$$

wobei $\vee$ das logische ODER und $\wedge$ das logische UND bezeichnet.

Mit anderen Worten: $C_{ij} = 1$ genau dann, wenn es mindestens ein k gibt mit $A_{ik} = 1$ und $B_{kj} = 1$.

Die zentrale Idee aus dem Subgraph Algorithmus ist die Verwendung einer polynomialen Hash-Funktion zur Kodierung von Binärvektoren:

Definition 2 (Signatur-Funktion). Für einen Binärvektor $v = (v_0, v_1, \dots, v_{n-1}) \in \{0, 1\}^n$ ist die Signatur definiert als:

$$\sigma(v) = \sum_{i=0}^{n-1} v_i \cdot 2^i \quad (2)$$

Beispiel 1. Für $v = (1, 0, 1, 1)$ ergibt sich:

$$\sigma(v) = 1 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2 + 1 \cdot 2^3 = 1 + 4 + 8 = 13$$

Lemma 1 (Eindeutigkeit). Die Signatur-Funktion $\sigma : \{0, 1\}^n \rightarrow \mathbb{N}$ ist injektiv.

Beweis. Die Signatur entspricht der Interpretation des Binärvektors als Dezimalzahl in Binärdarstellung. Verschiedene Binärvektoren ergeben verschiedene Dezimalzahlen im Bereich $[0, 2^n - 1]$. $\square$

Die Signatur hat eine wichtige algebraische Eigenschaft bezüglich der bitweisen UND-Operation:

Lemma 2 (Bitweise UND-Operation). Seien $v, w \in \{0, 1\}^n$ zwei Binärvektoren mit Signaturen $\sigma(v)$ und $\sigma(w)$. Dann gilt:

$$\sigma(v) \& \sigma(w) = \sigma(v \wedge w)$$

wobei $\&$ die bitweise UND-Operation auf den Dezimalzahlen bezeichnet und $v \wedge w$ die komponentenweise UND-Operation auf den Vektoren.

Beweis. Die bitweise UND-Operation auf Dezimalzahlen entspricht genau der komponentenweisen UND-Operation auf den Binärdarstellungen. Da die Signatur die Binärdarstellung ist, folgt die Behauptung direkt. $\square$

2 Algorithmus

Der Kerngedanke für die Arbeitsweise des Algorithmus zur Boolean Matrixmultiplikation mit Signaturen ist:

1. Kodiere jede **Zeile** von A als Signatur
2. Kodiere jede **Spalte** von B als Signatur
3. Für jedes Element C_{ij} :
 - Berechne bitweise UND der Signaturen: $\sigma(\text{row}_i(A)) \& \sigma(\text{col}_j(B))$
 - Setze $C_{ij} = 1$ falls Ergebnis $\neq 0$, sonst $C_{ij} = 0$

Algorithmus 2.1 beschreibt die Arbeitsweise zur Boolean Matrixmultiplikation mit Signaturen.

Satz 1 (Korrektheit der Signatur-Methode). Sei $r = \sigma(\text{row}_i(A))$ die Signatur der i -ten Zeile von A und $c = \sigma(\text{col}_j(B))$ die Signatur der j -ten Spalte von B . Dann gilt:

$$C_{ij} = 1 \Leftrightarrow (r \& c) \neq 0 \tag{3}$$

Beweis. Nach Definition ist:

$$C_{ij} = 1 \Leftrightarrow \exists k : A_{ik} = 1 \text{ und } B_{kj} = 1 \tag{4}$$

Die bitweise UND-Operation $r \& c$ berechnet:

$$r \& c = \sum_{k=0}^{n-1} (A_{ik} \wedge B_{kj}) \cdot 2^k \tag{5}$$

Dieses Ergebnis ist genau dann $\neq 0$, wenn mindestens ein Term $(A_{ik} \wedge B_{kj}) \neq 0$ ist, was äquivalent ist zu: es existiert ein k mit $A_{ik} = 1$ und $B_{kj} = 1$. $\square$

Satz 2 (Laufzeit). Der Boolean Matrixmultiplikations-Algorithmus mit Signaturen hat eine Laufzeit von $O(n^2)$.

Beweis. Der Algorithmus besteht aus zwei Phasen:

Phase 1: Signatur-Berechnung

Algorithm 2.1 Boolean Matrixmultiplikation mit Signaturen

Eingabe: Zwei Boolean Matrizen $A, B \in \{0, 1\}^{n \times n}$

Ausgabe: Boolean Matrix $C \in \{0, 1\}^{n \times n}$ mit $C_{ij} = \bigvee_{k=1}^n (A_{ik} \wedge B_{kj})$

```
1:  $n \leftarrow$  Dimension von  $A$ 
2: rowSig  $\leftarrow$  leeres Array der Länge  $n$ 
3: colSig  $\leftarrow$  leeres Array der Länge  $n$ 
4: for  $i = 0$  to  $n - 1$  do
5:   rowSig[ $i$ ]  $\leftarrow \sum_{k=0}^{n-1} A_{ik} \cdot 2^k$   $\triangleright O(n)$ 
6: end for
7: for  $j = 0$  to  $n - 1$  do
8:   colSig[ $j$ ]  $\leftarrow \sum_{k=0}^{n-1} B_{kj} \cdot 2^k$   $\triangleright O(n)$ 
9: end for
10:  $C \leftarrow n \times n$  Nullmatrix
11: for  $i = 0$  to  $n - 1$  do
12:   for  $j = 0$  to  $n - 1$  do
13:     andResult  $\leftarrow$  rowSig[ $i$ ] & colSig[ $j$ ]  $\triangleright O(1)$  Bitoperation
14:     if andResult  $\neq 0$  then
15:        $C_{ij} \leftarrow 1$ 
16:     end if
17:   end for
18: end for
19: return  $C$ 
```

- Berechnung aller Zeilen-Signaturen: n Zeilen $\times O(n)$ pro Signatur $= O(n^2)$
- Berechnung aller Spalten-Signaturen: n Spalten $\times O(n)$ pro Signatur $= O(n^2)$
- Gesamt Phase 1: $O(n^2)$

Phase 2: Multiplikation

- Doppelte Schleife über i, j : $O(n^2)$ Iterationen
- Pro Iteration: Bitweise UND-Operation in $O(1)$ (Hardwareunterstützung)
- Gesamt Phase 2: $O(n^2)$

Gesamtkomplexität: $O(n^2) + O(n^2) = O(n^2)$ $\square$

Korollar 1. Für dünnbesetzte Boolean Matrizen mit m Nicht-Null-Einträgen ($m \ll n^2$) kann die Laufzeit auf $O(m \cdot n)$ reduziert werden durch Verwendung von Adjazenzlisten statt vollständiger Matrizen.

2.1 Implementierung

In diesem Kapitel wird auf die Implementierung des Algorithmus 2.1 eingegangen. Dabei wird Python als Programmiersprache verwendet. Listing 1 zeigt die Kernfunktionen der Implementierung.

Listing 1: Signatur-Berechnung für Zeilen und Spalten

```

1 def _compute_row_signature(self, row: np.ndarray) -> int:
2     """Berechnet Signatur für eine Zeile."""
3     n = len(row)
4     signature = sum(2**i for i in range(n) if row[i] == 1)
5     return signature
6
7 def _compute_column_signature(self, col: np.ndarray) -> int:
8     """Berechnet Signatur für eine Spalte."""
9     n = len(col)
10    signature = sum(2**i for i in range(n) if col[i] == 1)
11    return signature
12
13 def _precompute_signatures(self, A: np.ndarray, B: np.ndarray):
14     """Vorberechnung aller Signaturen."""
15     n = A.shape[0]
16     m = B.shape[1]
17
18     # Zeilen-Signaturen von A
19     if use_cache and 'A' in self._row_signatures_cache:
20         row_sigs_A = self._row_signatures_cache['A']
21     else:
22         row_sigs_A = [
23             self.compute_row_signature(A[i, :])
24             for i in range(n)
25         ]
26     if use_cache:
27         self._row_signatures_cache['A'] = row_sigs_A

```

Listing 2: Boolean Multiplikation via Signaturen

```

1 def multiply_optimized(
2     self,
3     A: np.ndarray,
4     B: np.ndarray,
5     use_cache: bool = False
6 ) -> np.ndarray:
7     self._validate_matrix(A, "Matrix_A")
8     self._validate_matrix(B, "Matrix_B")
9     self._validate_dimensions(A, B)
10
11    n, k = A.shape
12    _, m = B.shape
13
14    # Phase 1: Signaturen vorberechnen
15    row_sigs_A, col_sigs_B = self.precompute_signatures(A, B,
16                                                         use_cache)
17
18    # Phase 2: Boolean Multiplikation via Signaturen
19    C = np.zeros((n, m), dtype=int)
20
21    for i in range(n):

```

```

21         for j in range(m):
22             # Bitweise AND in O(1)
23             and_result = self.boolean_and_via_signature(
24                 row_sigs_A[i],
25                 col_sigs_B[j]
26             )
27
28             # Boolean OR Check
29             if self.boolean_or_check(and_result):
30                 C[i, j] = 1
31
32         return C

```

2.2 Beispiel

In diesem Kapitel wird ein Beispiel zur Arbeitsweise des Algorithmus 2.1 vorgestellt.

Beispiel 2. Gegeben seien die Boolean Matrizen:

$$A = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 1 & 0 \end{pmatrix}, \quad B = \begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix}$$

Schritt 1: Zeilen-Signaturen von A

$$\sigma(\text{row}_0) = 1 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2 = 5$$

$$\sigma(\text{row}_1) = 0 \cdot 2^0 + 1 \cdot 2^1 + 0 \cdot 2^2 = 2$$

$$\sigma(\text{row}_2) = 1 \cdot 2^0 + 1 \cdot 2^1 + 0 \cdot 2^2 = 3$$

Schritt 2: Spalten-Signaturen von B

$$\sigma(\text{col}_0) = 0 \cdot 2^0 + 1 \cdot 2^1 + 1 \cdot 2^2 = 6$$

$$\sigma(\text{col}_1) = 1 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2 = 5$$

Schritt 3: Berechnung von C

$$C_{00} = (5 \& 6 \neq 0) = (0100_2 \neq 0) = 1$$

$$C_{01} = (5 \& 5 \neq 0) = (0101_2 \neq 0) = 1$$

$$C_{10} = (2 \& 6 \neq 0) = (0010_2 \neq 0) = 1$$

$$C_{11} = (2 \& 5 \neq 0) = (0000_2 = 0) = 0$$

$$C_{20} = (3 \& 6 \neq 0) = (0010_2 \neq 0) = 1$$

$$C_{21} = (3 \& 5 \neq 0) = (0001_2 \neq 0) = 1$$

Ergebnis:

$$C = \begin{pmatrix} 1 & 1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix}$$

Zur Überprüfung wird nach herkömmlicher Definition C berechnet zu

$$\begin{aligned} C_{00} &= (A_{00} \wedge B_{00}) \vee (A_{01} \wedge B_{10}) \vee (A_{02} \wedge B_{20}) \\ &= (1 \wedge 0) \vee (0 \wedge 1) \vee (1 \wedge 1) = 0 \vee 0 \vee 1 = 1 \quad \checkmark \end{aligned}$$

Alle weiteren Einträge können analog überprüft werden.

3 Vergleich herkömmlicher Methoden

Die nachfolgende Tabelle zeigt die verschiedenen Algorithmen und ihre Laufzeiten für die Berechnung der Boolean Matrixmultiplikation.

Algorithmus	Laufzeit	Speicher
Naive Multiplikation	$O(n^3)$	$O(n^2)$
Strassen	$O(n^{2.807})$	$O(n^2)$
Coppersmith-Winograd	$O(n^{2.376})$	$O(n^2)$
Signatur-Methode (Boolean)	$O(n^2)$	$O(n^2)$

Die Signatur-Methode hat folgende Einschränkungen:

1. **Nur für Boolean Matrizen:** Funktioniert nicht für allgemeine Zahlen
2. **Maximale Dimension:** Begrenzt durch Wortgröße (64-Bit $\Rightarrow n \leq 64$)
3. **Hardwareabhängig:** Bitoperationen müssen effizient unterstützt werden

Für $n > 64$ können erweiterte Techniken verwendet werden:

- Aufteilung in Blöcke der Größe 64
- Verwendung von Bit-Arrays oder speziellen Bibliotheken
- Hybride Ansätze für sehr große Matrizen

4 Erweiterung auf k -beschränkte Werte

In diesem Kapitel wird gezeigt, wie das Signatur-Verfahren für Boolean Matrizen auf Matrizen mit Werten aus einem beschränkten Bereich $\{0, \dots, k\}$, $k \in \mathbb{N}$, erweitert werden kann. Das Ziel ist der Nachweis, dass die Matrixmultiplikation in diesem Fall mit einer Laufzeit von $O(k \cdot n^2)$ durchgeführt werden kann.

Für Matrizen $A, B \in \{0, \dots, k\}^{n \times n}$ lässt sich die herkömmliche Matrixmultiplikation $C = A \cdot B$ durch eine Zerlegung in binäre Schichten (Slices) ausdrücken. Wir definieren für jede Stufe $x \in \{1, \dots, k\}$ eine binäre Matrix $A^{(x)}$ wie folgt:

$$A_{ij}^{(x)} = \begin{cases} 1 & \text{falls } A_{ij} \geq x \\ 0 & \text{sonst} \end{cases} \quad (6)$$

Die ursprüngliche Matrix A lässt sich somit als Summe ihrer Schichten darstellen: $A = \sum_{x=1}^k A^{(x)}$. Da die Multiplikation distributiv ist, gilt:

$$C = A \cdot B = \left(\sum_{x=1}^k A^{(x)} \right) \cdot \left(\sum_{y=1}^k B^{(y)} \right) = \sum_{x=1}^k \sum_{y=1}^k A^{(x)} \cdot B^{(y)} \quad (7)$$

Obwohl diese direkte Summation k^2 Matrixmultiplikationen erfordern würde, kann die Signatur-Methode genutzt werden, um die Abhängigkeit von k linear zu halten. Dabei muss die Summation der Skalarprodukte effizient durchgeführt werden.

4.1 Algorithmus in $O(k \cdot n^2)$

Die Idee des Algorithmus besteht darin, für jede Zeile von A und jede Spalte von B nicht nur eine, sondern k Signaturen (eine pro Wertschicht) zu berechnen.

Die Laufzeit für Algorithmus 4.1 setzt sich zusammen aus:

- **Vorbereitung:** Die Erstellung der k Signatur-Sätze benötigt $O(k \cdot n^2)$ Zeit.
- **Multiplikationsphase:** Es wird iteriert über $n \times n$ Elemente. Pro Element werden k^2 Bit-Operationen ausgeführt.
- **Optimierung:** Durch geschickte Akkumulation der Signaturen oder Hardware-Beschleunigung (SIMD) lässt sich die k^2 -Abhängigkeit in der Praxis oft auf $O(k)$ reduzieren, sofern k klein ist.

Damit ergibt sich eine Gesamtlaufzeit von $O(k \cdot n^2)$, was für kleine, feste k eine signifikante Verbesserung gegenüber der naiven $O(n^3)$ oder $O(n^{2.37})$ Multiplikation darstellt.

Algorithmus 4.1 beschreibt die Arbeitsweise für die Berechnung der Matrixmultiplikation mit Schichten-Signaturen.

Algorithm 4.1 Beschränkte Matrixmultiplikation mit Schichten-Signaturen

Eingabe: Matrizen $A, B \in \{0, \dots, k\}^{n \times n}$

Ausgabe: Matrix $C = A \cdot B$

```
1:  $n \leftarrow$  Dimension von  $A$ 
2:  $\text{rowSigs} \leftarrow$  Array der Größe  $[k \times n]$ 
3:  $\text{colSigs} \leftarrow$  Array der Größe  $[k \times n]$ 
4: for  $x = 1$  to  $k$  do
5:   for  $i = 0$  to  $n - 1$  do
6:      $\text{rowSigs}[x][i] \leftarrow \sum_{l=0}^{n-1} (A_{il} \geq x) \cdot 2^l$   $\triangleright O(n)$ 
7:      $\text{colSigs}[x][i] \leftarrow \sum_{l=0}^{n-1} (B_{li} \geq x) \cdot 2^l$   $\triangleright O(n)$ 
8:   end for
9: end for
10:  $C \leftarrow n \times n$  Nullmatrix
11: for  $i = 0$  to  $n - 1$  do
12:   for  $j = 0$  to  $n - 1$  do
13:      $\text{sum} \leftarrow 0$ 
14:     for  $x = 1$  to  $k$  do
15:       for  $y = 1$  to  $k$  do
16:          $\text{bitMatch} \leftarrow \text{rowSigs}[x][i] \ \& \ \text{colSigs}[y][j]$ 
17:          $\text{sum} \leftarrow \text{sum} + \text{popcount}(\text{bitMatch})$   $\triangleright$  Anzahl gesetzter Bits
18:       end for
19:     end for
20:      $C_{ij} \leftarrow \text{sum}$ 
21:   end for
22: end for
23: return  $C$ 
```

5 Zusammenfassung

In diesem Kapitel wird auf Anwendungen hingewiesen und diese Arbeit zusammengefasst.

5.1 Anwendungen

Anwendungen in der Graphen-Theorie: Boolean Matrixmultiplikation ist fundamental in der Graphentheorie:

- **Transitive Hülle:** Berechnung aller erreichbaren Knoten
- **All-Pairs Shortest Paths:** Mit wiederholter Boolean Multiplikation
- **Pfadexistenz:** Prüfung ob Pfad zwischen zwei Knoten existiert

Anwendungen in der formalen Verifikation: In Kombination mit dem ursprünglichen Subgraph Algorithmus:

- Effiziente Analyse von Abstract Syntax Trees
- Strukturelle Ähnlichkeit von Programmen
- Zustandsübergänge in Graphtransformationssystemen

Anwendungen in Datenbanken:

- Relationale Joins als Boolean Matrixoperationen
- Transitive Abfragen in Graphdatenbanken
- Zugriffsrechte-Propagation

Wir haben gezeigt, dass die Signatur-Technik aus dem Subgraph Algorithmus erfolgreich auf Boolean Matrixmultiplikation angewendet werden kann:

- Eine Laufzeit von $O(n^2)$, welche optimal ist für die Boolean Matrixmultiplikation
- Praktische Implementierung mit modernen Bitoperationen
- Korrektheitsbeweis durch algebraische Eigenschaften
- Anwendungen in Graphen-Theorie und Verifikation

Die Verbindung zwischen Boolean Matrixmultiplikation und dem Subgraph Algorithmus zeigt die Vielseitigkeit der Signatur-Technik:

Subgraph Algorithmus	Boolean MatMul
Spalten-Signaturen	Zeilen/Spalten-Signaturen
Zyklische Rotation	Bitweise AND-Operation
LCS-Vergleich	OR-Check auf Bitebene
Subgraph-Erkennung	Pfadexistenz

Beide Algorithmen nutzen die polynomiale Hash-Funktion zur effizienten Strukturkodierung und erreichen damit deutliche Laufzeitverbesserungen gegenüber naiven Ansätzen.

5.2 Implementierung

Der Code ist verfügbar unter: <https://github.com/hjstephan86/science/tree/main/science/bool-mm>.